ACPC Notes — Contents

1 Contest Playbook **1**

1.1 ACPC, Kattis, and Team Strategy 1

1.2 Template, Debugging, Stress Testing 2

1.3 When You're Stuck 3

1.4 ACPC 2016–2024: Every Problem → Technique 6

2 Core Techniques **9**

2.1 Binary Search, Ternary Search, Parametric Search 9

2.2 Prefix Sums, Difference Arrays, Sweep Line, Contribution 10

2.3 Two Pointers, Monotonic Stack, Monotonic Deque 11

2.4 Greedy and Exchange Arguments 12

2.5 Complete Search, Meet in the Middle, Square Root, Randomization 13

2.6 Bit Manipulation and Bitset 15

1. Contest Playbook

1.1 ACPC, Kattis, and Team Strategy

How to use these notes. Stuck on a problem → go to *When You're Stuck* (§1.3): constraint → complexity table, then keyword → technique table, which points to the section with code. Every C++ block compiles as-is after the template (§1.2) with `#include <bits/stdc++.h>`, `using namespace std;`, `using ll = long long;`. Blocks marked **[stress-tested]** were checked against brute force; the rest compile and were hand-verified, so still test on samples.

1.1.1 What ACPC looks like (from all sets 2016–2024 on Kattis)

- 8–15 problems per set. Kattis difficulty (1–10 scale) spread is very consistent:
- **~35% easy (1.2–2.9):** implementation, careful reading, simulation. Solve these *fast and without WA* — penalty time decides ranks among teams with equal solves.
- **~35% medium (3.0–5.0):** one standard idea: binary search on answer + greedy, monotonic stack, Dijkstra/BFS with a twist, tree DP, counting mod p, interval DP, grid flood fill, offline DSU.
- **~30% hard (5.1–8.3):** probability/expected-value DP, flows with modelling (min flow with lower bounds, assignment), FFT, number theory (factoring, gcd tricks), interactive problems with tiny query budgets, geometry + search, string canonical forms, nasty interval DP.
- Expert/CM level: easy+medium ≈ CF Div2 A–C; hard ≈ Div2 D–F / Div1 C–D. Hard ACPC problems are usually *one known technique plus modelling* rather than research-level tricks — recognizing the model is the whole game. See §1.4 for every past problem mapped to its technique.
- Themes recur: Kattis "Medium/Hard" problems from ACPC very often involve **probability**, **flows/matching**, **Dijkstra on augmented state**, **binary search on the answer**, and **number theory**.

1.1.2 Kattis specifics

- Language: C++ compiled with GCC, `-O2`, gnu++17 or newer, 64-bit. `__int128`, `__builtin_*`, `<bits/stdc++.h>`, `__gnu_pbds` all work.
- Time limit is **per test file**; there may be dozens of files. Memory is usually 1024 MB (check the problem page).
- Verdicts stop at the first failing test file — you don't learn which case failed. A WA on test 1 means the sample itself fails (or format is wrong).
- Output: trailing whitespace/newline differences are ignored. Floating answers are accepted within the stated absolute/relative error; print with `fixed << setprecision(10)`.
- **Interactive problems** (ACPC 2024 had two): flush after every query (`cout << ... << endl` or `cout.flush()`), never read past what the judge sends, terminate immediately after the final answer. See §10.1.
- Multiple test cases inside one file (`T` given): **reset all global state**.
- Python: big integers are trivial in Python; consider it for one-off big-number problems (e.g. ACPC 2017 *Divide by 100* has numbers with 10^6 digits — just string manipulation though).
- Recursion: Kattis has a generous stack, but DFS depth 10^6 in C++ is still risky; prefer iterative for path graphs with $n \geq 10^6$.

1.1.3 Team strategy (2–3 people, one machine)

1. **First 15 min:** split the problem set; everyone reads, labels each problem `easy / medium / hard / unknown` and writes one-line summary + constraints on paper. Read *all* problems — ACPC difficulty is not ordered by letter.
2. **Keyboard discipline:** only one person types; others think on paper. Never debug at the keyboard while someone with a ready solution waits — print/write the code and debug on paper.
3. Watch the scoreboard: problems solved by many teams are easier than they look. If 10 teams solved X and you think it's hard, you're missing something simple.
4. **Before submitting:** run all samples, a tiny edge case ($n = 1$, all equal, max values), and check output format. One WA costs 20 minutes of penalty.
5. **After WA:** reread the statement *first* (most WAs are misreads), then check §1.3 bug list, then write a brute force and compare on random small inputs.
6. **Last hour:** stop starting new hard problems unless the idea is complete; finish/debug what's closest. Freeze-time submissions still count.
7. Two people can pair on the hardest problem: one explains the problem from scratch to the other — this exposes wrong assumptions.

1.1.4 Precision and number sizes (memorize)

type	max	notes
`int`	$2.1 \cdot 10^9$	`a*b` overflows when both $\sim 5 \cdot 10^4$
`long long`	$9.2 \cdot 10^{18}$	product of two 10^9 values fits; two 10^{10} doesn't
`__int128`	$1.7 \cdot 10^{38}$	multiply two `ll` safely; no I/O, write a printer
`double`	15–16 digits	integers exact up to $2^{53} \approx 9 \cdot 10^{15}$
`long double`	18–19 digits	80-bit on x86 GCC; slower but helps precision

- Operations/second: $\sim 3 \cdot 10^8$ simple ops in 1 s with `-O2`; `set/map` $\sim 10^7$ ops/s; `unordered_map` $\sim 3 \cdot 10^7$ with reserve; `cin` with sync off $\sim 10^7$ ints/s.
- Memory: `int` array of 10^8 = 400 MB. `vector<vector<int>>` with 10^6 small vectors has big overhead — use CSR / flat arrays.

1.2 Template, Debugging, Stress Testing

1.2.1 Main template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "] = ", _dbg(__VA_ARGS__)
template <class T> void _dbg(const T& x) { cerr << x << endl; }
template <class T, class... R> void _dbg(const T& x, const R&... r) {
    cerr << x << ", "; _dbg(r...);
}
#else
#define dbg(...)
#endif
const ll INF = 4e18;  // careful: INF + INF overflows. Use 1e18 if you add two.
const int MOD = 1e9 + 7;

void solve() {
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int T = 1;
    // cin >> T;
    while (T--) solve();
}
```

Compile locally: `g++ -std=gnu++17 -O2 -Wall -Wextra -DLOCAL -fsanitize=address,undefined -g a.cpp`. Sanitizers catch out-of-bounds and signed overflow immediately — use them whenever you get RE/WA.

1.2.2 Useful snippets

```cpp
// print __int128
void print128(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar('0' + (int)(x % 10));
}
// floor/ceil division that work for negatives
ll fdiv(ll a, ll b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); }
ll cdiv(ll a, ll b) { return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); }
// exact integer sqrt (floor)
ll isqrt(ll n) { ll r = sqrtl((ld)n); while (r * r > n) --r; while ((r + 1) * (r + 1) <= n) ++r; return r; }
// random
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(rng); }
// anti-hack hash for unordered_map<ll, V, chash>
```

```cpp
struct chash {
    static ull sm(ull x) { x += 0x9e3779b97f4a7c15; x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb; return x ^ (x >> 31); }
    size_t operator()(ull x) const { static const ull R = rng(); return sm(x + R); }
};
// hash a pair: use key = (ll)a << 32 | (unsigned)b
```

- Read until EOF: `while (cin >> x)`. Read a line: `getline(cin, s)`; after `cin >> n` call `cin.ignore()` (or `getline` once) before the first `getline`.
- Output many lines: use `'\n'`, never `endl` (except interactive).
- Doubles: `cout << fixed << setprecision(10) << x;` — avoid printing `-0.000000` (add `+ 0.0` or check `abs(x) < eps`).
- Sorting with comparator must be a *strict weak ordering* (`<`, never `<=`), otherwise RE/UB.
- `lower_bound(all(v), x)` → first `>= x`; `upper_bound` → first `> x`. Count of `x` in sorted v: `upper - lower`.
- `set.lower_bound(x)` is O(log n); `lower_bound(all(set), x)` is **O(n)**. Same for multiset/map.
- `multiset.erase(x)` removes **all** copies of x; use `ms.erase(ms.find(x))`.
- `priority_queue` is a max-heap; min-heap: `priority_queue<T, vector<T>, greater<T>>`.
- `accumulate(all(v), 0LL)` — the `0LL` matters.
- `(int)v.size() - 1` — `v.size() - 1` is unsigned and wraps when empty.
- `1 << k` is `int`; use `1LL << k` for k ≥ 31. `__builtin_popcountll` for `ll`.
- `memset` only for 0 / -1 / `0x3f` (int ≈ 1.06e9, sum of two still fits) .

1.2.3 Stress testing (do this after the first unexplained WA)

```bash
# gen.py prints a random small test; brute.cpp is the obviously-correct slow solution
g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp
for i in $(seq 1 1000); do
  python3 gen.py $i > in.txt
  ./sol < in.txt > o1.txt; ./brute < in.txt > o2.txt
  if ! cmp -s o1.txt o2.txt; then echo "DIFF on seed $i"; cat in.txt; break; fi
done
```

```python
# gen.py
import random, sys
random.seed(int(sys.argv[1]))
n = random.randint(1, 6)
print(n)
print(*[random.randint(1, 10) for _ in range(n)])
```

Keep tests *tiny* (n ≤ 6, values ≤ 10): the smallest failing case is the easiest to debug by hand.

1.3 When You're Stuck

1.3.1 Constraints → target complexity

Budget ≈ $3 \cdot 10^8$ simple operations per second (less with maps/sets, recursion, cache misses).

n up to	target	typical techniques
8–11	O(n!·n)	all permutations (`next_permutation`), brute force
15–20	$O(2^n \cdot n)$, $O(3^n)$ at 15	bitmask DP, subset enumeration, submask DP
35–45	$O(2^{n/2})$	meet in the middle
50–100	$O(n^4)$, $O(n^3 \log)$	Floyd, flows, small Hungarian, DP with 3–4 dims
300–500	$O(n^3)$	interval DP, Floyd, Gaussian elimination, Hungarian
2000–5000	$O(n^2)$	2D DP, all pairs, $O(n^2)$ with bitset /32 for 10^4–10^5
10^5–$2 \cdot 10^5$	O(n log n), $O(n \log^2 n)$, O(n√n)	sort, BIT/segtree, binary search, Dijkstra, Mo's
10^6	O(n), O(n log n) small const	linear scans, sieve, prefix sums, two pointers, KMP/Z
10^9–10^{18}	O(log n), O(√n), O(cube-root)	math formula, binary search, matrix power, digit DP, factorization

- **Sum of n over tests ≤ X** → per-test complexity can use X. Otherwise multiply by T!
- Two small parameters (e.g. n ≤ 10^5, k ≤ 20) → O(n·k) or $O(n \cdot 2^k)$ DP is intended (ACPC 2022 *City Numbers*: k ≤ 20 colours on a tree).

- A weirdly specific bound is a hint: `n ≤ 15` (bitmask/interval brute), `values ≤ 200` (enumerate the value — ACPC 2018 *Dog Trouble*), `prime factors < 10^6` (trial division works), `p < 10^10, s ≤ 10^5` (s^2 can exceed p).
- Output modulo a prime → counting, combinatorics, DP; answer would be huge otherwise.
- Output a real number with 1e-6 → binary/ternary search on reals, geometry, probability.

1.3.2 Keyword → technique

if the problem says / looks like ...	try	where
minimize the maximum / maximize the minimum	binary search on answer + greedy/BFS check	§2.1
"find the smallest X such that feasible" and feasibility is monotone	binary search on X	§2.1
maximize ratio Σa/Σb	binary search λ: is max Σ(a − λb) ≥ 0 (Dinkelbach)	§2.1
unimodal function of a real variable	ternary search / golden section	§2.1
k-th smallest of something implicit	binary search on value, count ≤ mid	§2.1
subarray sums/xors, count subarrays with property	prefix sums + hashmap / two pointers	§2.2, §2.3
range add many times, query once	difference array	§2.2
sum over all pairs/subarrays of f	contribution technique: count how often each element contributes	§2.2
nearest greater/smaller, "visible", histogram	monotonic stack	§2.3
sliding window min/max	monotonic deque	§2.3
schedule jobs with deadlines/durations	sort by deadline + heap; exchange argument	§2.4
choose k things to maximize something, n large	greedy + heap; check with exchange argument	§2.4
n ≤ 40, subset sums	meet in the middle	§2.5
connectivity under edge deletions, offline	reverse time → DSU additions	§3.1
point update + range query	Fenwick / segment tree	§3.2–3.3
range update + range query	lazy segment tree	§3.3
static range min/gcd	sparse table	§3.4
k-th element in dynamic set, order of key	PBDS ordered set / BIT walk	§3.4
many range queries, offline, no updates, hard to merge	Mo's algorithm	§3.4
count / optimize over sequences, choices at each step	DP; define state = what you need to know about the past	§4.1
subsequence counting (e.g. palindromic subsequences)	interval DP with inclusion–exclusion	§4.1
interleaving two strings, edit distance, LCS	2D DP	§4.1
merge adjacent / parenthesize / split interval	interval DP $O(n^3)$; keep max AND min if signs	§4.1
n ≤ 20 items, choose order/assignment	bitmask DP	§4.2
count numbers in [L, R] with digit property	digit DP	§4.2
tree, choose labels/colours per node with neighbour constraint	tree DP (keep best & second best)	§4.3
answer for every root	rerooting DP	§4.3
dp[i] = min_j (dp[j] + b_j·a_i)	convex hull trick / Li Chao	§4.4
dp[k][i] = min_j dp[k−1][j] + cost(j,i), cost "Monge"	divide & conquer opt / Knuth	§4.4
expected value, probability of winning, optimal stopping	linearity of expectation; DP over states backward	§4.5
turns repeat in a cycle with probabilities	geometric series / solve linear equations	§4.5
two players, optimal play, who wins	win/lose DP, Sprague–Grundy, find pattern by brute force	§4.5
shortest path, unweighted / weights 0–1 / nonneg	BFS / 0-1 BFS / Dijkstra	§5.1
shortest path with extra condition (fuel, horse, key, parity)	Dijkstra on (node, state)	§5.1
distance to nearest of many sources	multi-source BFS/Dijkstra	§5.1
path maximizing the minimum edge/vertex value	binary search + BFS, or max spanning tree, or modified Dijkstra	§5.1
which sums are representable by coins (unbounded)	Dijkstra on residues mod smallest coin	§5.1
minimum ratio spanning tree	binary search λ + MST	§5.2
dependencies / ordering	topological sort, DAG DP	§5.2
each node has one outgoing edge	functional graph: cycles + trees, binary lifting	§5.2
boolean constraints "a or b"	2-SAT	§5.3

if the problem says / looks like ...	try	where
edges whose removal disconnects	bridges / articulation points	§5.3
use every edge exactly once	Euler path (Hierholzer)	§5.3
tree path queries, ancestors, distances	LCA / binary lifting / Euler tour + BIT / HLD	§5.4
count paths in tree with property	centroid decomposition / small-to-large	§5.4
assign workers to jobs one-to-one, min cost	Hungarian / min-cost flow	§6.3
max pairs with compatibility	bipartite matching	§6.2
min #paths covering all vertices of a DAG	n − max matching (split graph)	§6.2
min #paths covering all **edges** of a DAG	min flow with lower bounds = Dilworth on edges	§6.1
cut with costs, "choose set with dependencies, max profit"	min cut / project selection	§6.1
grid with dominoes / chessboard parity	bipartite matching on black/white cells	§6.2
gcd, lcm, divisibility, a·x + b·y = c	ext. gcd, CRT	§7.1–7.2
sum of divisors, factorization of big n	sieve / trial division up to √n / Pollard rho	§7.2
count arrangements, multisets, paths	stars & bars, Catalan, inclusion–exclusion, Burnside	§7.3
linear recurrence with huge n	matrix exponentiation	§7.4
system of linear (mod 2) equations, max xor subset	Gaussian elimination / xor basis	§7.4
count pairs with sum = k for all k, pattern matching counts	FFT / NTT convolution	§7.5
string equality of substrings fast	polynomial hashing	§8.1
pattern occurrences / borders / periods	KMP prefix function, Z-function	§8.1
rotations / cyclic shifts equal	minimal rotation (Booth) as canonical form	§8.2
longest palindrome substring	Manacher	§8.2
many patterns in a text	Aho–Corasick	§8.2
distinct substrings, LCP of suffixes	suffix array + LCP	§8.2
polygon area, orientation, lattice points	shoelace, cross product, Pick	§9.1–9.2
smallest enclosing / outer boundary of points	convex hull	§9.2
interactive: judge answers queries	binary search / information-theoretic planning / gcd tricks	§10.1
evaluate expressions, mini-language	recursive descent parser	§10.2

1.3.3 Stuck-on-an-idea checklist

1. **Re-read the statement.** Underline every constraint. Is something guaranteed (distinct values, connected, sorted, a < b)? That guarantee is usually *used*.
2. **Solve tiny cases by hand** (n = 1, 2, 3) and **write a brute force**; print answers for n = 1..10 and look for a pattern (then search OEIS-style in your head: powers of 2, Catalan, Fibonacci, n^2, C(2n,n)).
3. **What is the answer monotone in?** If "answer ≥ x is feasible" is monotone → binary search. Fixing the answer often turns optimization into a yes/no greedy.
4. **Fix one thing and iterate over it:** fix the maximum element, the left endpoint, the last chosen item, the root, the value of one variable (e.g. enumerate the max time t ≤ 200 then solve the rest).
5. **Sort** the input (by start, end, value, ratio). Many greedy/DP problems become easy after sorting by the right key; find the key with an exchange argument.
6. **Reverse** something: process time backward (deletions → additions), reverse the graph, think about the complement ("at least one" = total − "none").
7. **Change what you iterate over:** over values instead of indices, over divisors instead of multiples (harmonic sum n log n), over the answer instead of the input.
8. **Model as a graph:** states = nodes, moves = edges. Grid + extra state (direction, keys, used power-up) → BFS/Dijkstra on product graph.
9. **Model as a flow/matching** when there are "pairings", "assign each X to one Y", "each used at most once", "minimum number of chains/paths".
10. **Linearity of expectation:** E[sum] = Σ E[indicator]. Never compute the full distribution if only an expectation is asked.
11. **Bound the search:** the answer is small (≤ 60 when doubling, ≤ log n), the number of distinct values is small (≤ √(2n) distinct lengths if Σ = n), only O(√n) distinct `n / i` values.
12. **Look for invariants** (parity, sum mod k, xor) in games and constructive problems.

13. Think about the **special structure:** tree (unique paths, DP), DAG (topo order), bipartite, planar grid, permutation (cycles), convex polygon ($O(n^3)$ DP over vertices).
14. Still stuck after 20–30 min with no progress → switch problems, come back later.

1.3.4 WA / RE / TLE checklist

- **Overflow**: `int * int`, sums of 10^5 values of 10^9, `1 << 40`, `INF + x`. When in doubt, `ll` everywhere.
- **Multi-test state** not reset (globals, adjacency lists, visited arrays, DSU).
- **Off-by-one**: inclusive vs exclusive ranges; 0- vs 1-indexed input; array size `n` vs `n + 1`.
- **Edge cases**: n = 1, empty set, all equal, all negative, answer 0, disconnected graph, self-loops, multi-edges, duplicate points, collinear points.
- **Reading**: multiple values per line, strings with spaces (`getline`), trailing spaces, Windows `\r`.
- **Output**: exact format/capitalisation ("Yes" vs "YES"), precision, one answer per line, `-0.0`.
- **Negative modulo**: `((a % m) + m) % m`. Subtraction in modular arithmetic: `(a - b + MOD) % MOD`.
- **Doubles compared with `==`**; use eps or exact integer arithmetic (cross products in `ll`).
- **Uninitialised** local variables / arrays (globals are zeroed, locals aren't).
- **Recursion depth** (10^6 deep DFS), **stack arrays** too big (`int a[10^7]` inside `main` → use global/vector).
- **TLE**: `endl` flushing, `map` where an array works, passing `vector` by value, O(n) `lower_bound` on `set`, `string +=` in a loop creating copies (`s = s + c` is O(n)), recomputing in loops, too many `long double`.
- **Dijkstra**: skip stale heap entries (`if (d > dist[u]) continue;`), else worst case is quadratic.
- **Comparator** not strict weak ordering → crash in `sort`.
- **Wrong algorithm**: greedy without proof → stress test against brute force.

1.4 ACPC 2016–2024: Every Problem → Technique

Kattis difficulty in brackets. Technique = the intended core idea (my analysis of the statements; `?` = likely but unverified). Use this to calibrate: what does a 4.0 vs 6.5 ACPC problem look like?

year	problem [diff]	core technique
2016	Mirror Images [1.6], Soylent [1.7], Bits [2.2]	implementation
	Wood Cutting [2.5]	sort ascending (shortest processing time first), prefix sums
	Dacey the Dice [3.3]	BFS on (cell, die orientation) — 24 orientations
	Flower Garden [3.6]	prefix of Manhattan distances + sieve for primes
	Palindromic Naming [4.1]	count palindromic subsequences, interval DP $O(n^2)$ mod p (§4.1)
	City Destruction [5.6]	linear DP over buildings with state "was neighbour destroyed before me"
	Modelling Problems [6.8]	parse a mini-language, brute force over unknown 0/1 values
2017	Musical Scales [1.6], Tetration [1.7]	implementation / math ($x = a^{1/a}$)
	Quantum Superposition [2.2]	DAG DP: set of reachable path lengths (bitset), combine two sets
	Anthony and Diablo [2.3]	geometry: max area for perimeter is a circle
	Divide by 100 [3.2]	big-number string manipulation
	Concentration [4.8]	probability / game simulation reasoning ?
	Lane Switching [5.5]	maximize the minimum (bottleneck path) over intervals: Dijkstra/BFS ?
	Maximal Sequences [6.2]	offline by start index; ordered set of next occurrences per value; check first m+1
	Matchings [7.5]	2-D pattern matching counts via FFT convolution, one FFT per colour (§7.5)
2018	Early Winter [2.0], Eating Out [2.5]	implementation / pigeonhole counting
	Arachnophobia [4.0]	multi-source Dijkstra from spiders + binary search D + Dijkstra s→t
	Dog Trouble [5.8]	enumerate max time t (≤ 200), assignment (Hungarian) maximizing Σt_i (§6.3)
	PUBNite [5.8]	geometry + binary/ternary search on time

year	problem [diff]	core technique
	Exploding Kittens [6.4]	simulation, skipping irrelevant cards ?
	Homework [6.5]	string interleaving DP O(len1·len2) (bitset if tight)
	Trimming Convex Polygons [6.9]	DP over convex polygon vertices $O(n^3)$ (fix first kept vertex, triangle fan)
2020	Test Drive [1.9], Laptop Stickers [1.9], Warring Scoring [2.8]	implementation
	Path Crossings [2.9]	hashing (x,y,t) / sort events
	Straights [2.9]	greedy on value counts
	Listen To Your Boss [3.9]	tree ancestor queries: Euler tour tin/tout (§5.4)
	Wormhole Extreme!!! [4.4]	kinematics; minimum initial velocity via v^2 recursion / binary search
	Hired Help [5.5]	binary search #employees + greedy deadline check
	Monochromatic Minesweeper [6.2]	constructive / search with pruning ?
	Password Rotation [6.3]	canonical form = min rotation of s and reverse(s) (Booth), hash set (§8.2)
	Wrapping Trees [7.3]	fractal → convex hull of the self-similar set; geometry
	Best Investing [7.8]	math / greedy over deposit–withdraw schedule ?
2021	Finding An A [1.2], GCVWR [1.4], Square Peg [1.5], Simone [1.8], Colour Wars [2.3]	implementation
	Snakes and Masters [2.9]	Fibonacci DP mod 10^6
	Bitmask [3.1]	string/bit matching with DP ?
	Menu Updates [3.3]	maintain smallest available number: set/heap
	Associative Exponents [3.4]	compare $a^{b\hat{}c}$ vs $(a^b)^c \Leftrightarrow b^c$ vs b·c, careful with overflow
	Ghostbusters 3 [3.8]	non-crossing assignment: sort + greedy/DP ?
	Wonky Pizza [4.2]	polar area $\int r^2/2\, d\theta$ closed form + binary search the angle
	Beep Code [4.6]	bit decomposition of a sum of square waves; reason level by level
	Balls and Bins [5.1]	optimal-stopping expected-value DP over (#bins with 1, #bins with 2) (§4.5)
	Swapping Places [8.1]	constructive: permutation routing on spanning tree, process leaves first
	Compression [8.2]	DP over block boundaries with careful cost precomputation
2022	Undead or Alive [1.3], Ordinals [1.8], Count Doubles [1.8]	implementation
	Contingency Planning [2.1]	$\Sigma_k\ n!/(n-k)!$ with cap
	Simple Factoring [2.4]	discriminant $b^2 - 4ac$ is a perfect square
	Maximum Fix [3.4]	each element votes for the one shift that fixes it; count array
	Supply Routes [3.5]	offline: reverse edge deletions → DSU unions (§3.1)
	Fight to Survive [3.8]	greedy / simulation ?
	City Numbers [4.4]	tree DP dp[v][colour], keep best & second best child value (§4.3)
	Long Walk [5.5]	implicit graph $i \leftrightarrow 2i \bmod 2^n$: bit manipulation, LCA-like on binary strings
	Braaaains [5.8]	Frobenius/coin reachability: Dijkstra on residues mod smallest size (§5.1)
	Tricky Factoring [6.7]	factor a, c (primes $< 10^6$), enumerate divisor pairs, count distinct b = dg + ef
	Maximal Parentheticals [8.0]	interval DP over expression tracking max and min (signs flip)
	Escape Routes [8.3]	min #paths covering all DAG edges = min flow with lower bounds (§6.1)
2023	Buildin' Fences [1.5]	implementation
	Flying Straight [3.0]	trigonometry (arc vs tangent)

year	problem [diff]	core technique
	Buildin' Roads [3.3]	minimum ratio spanning tree: binary search λ + MST (§2.1, §5.2)
	OBD Codes [3.9]	pattern matching on event streams ?
	Counting Cacti [4.0]	grid parsing + flood fill components
	Outlaw Numbers [4.0]	aliquot sequence: sum of proper divisors via factorization, ≤ 32 steps
	Wrangler Rating [4.5]	careful simulation of rules
	Circuit Game [5.2]	graph connectivity under switch removal + budget ?
	Dice Tables [5.2]	counting multisets (stars and bars) over overlapping face ranges, mod 10^9+9
	Desert Strike [5.4]	lattice counting of a rasterized rotated square (math)
	High Noon Language Showdown [5.5]	brute force over n with exact integer ceil(log2)
	Temporal Turmoil [6.7]	clock-hand alignment math with exact fractions
	Horse Snatchers [6.8]	binary search horse speed + Dijkstra on (node, has horse)
	Gunslingers [7.8]	probability DP over survivor sets with cyclic turns (geometric series)
2024	Snowfall [1.6], Melting Snow [1.7], Handheld Fan [2.2]	implementation
	Wrapping Snowballs [3.5]	?
	Alpine Agility [3.8]	grid DP (moves S/E/SE)
	Slippery Floor [3.8]	BFS where a move slides until hitting an obstacle
	Topographic Isolation [4.0]	monotonic stack: nearest $\geq$ on each side (§2.3)
	A Little to the Right [4.2]	count distinct sorted orders among properties with all-distinct values
	Hockey Fans [4.4]	binary search on D + greedy left-to-right placement
	Uniform Variation [4.7]	brute force: groups of all-different are ≤ 4; enumerate small subsets
	Snow Way Out [6.7]	interactive, 6 comparisons on 10×10 grid: offline minimax search for a strategy
	Four Questions [7.1]	interactive: $s^2 \bmod p \Rightarrow p$ divides $s^2 - r$; gcd of such values (§10.1)
	Speedy Slopes [7.2]	shortest path with speed state on grid ?

Patterns to take away

- Binary search on the answer appears almost every year (Arachnophobia, Hired Help, Hockey Fans, Horse Snatchers, Buildin' Roads).
- Dijkstra variants: multi-source, state-augmented, residue graph (Braaaains).
- Probability/expected value DP appears most years at 5–8 difficulty.
- One flow/matching problem every year or two (Dog Trouble, Escape Routes).
- Interactive problems appeared in 2024 (two of them) — read §10.1 before the contest.
- Number theory with "all prime factors $< 10^6$" or big-number twists.

2. Core Techniques

2.1 Binary Search, Ternary Search, Parametric Search

2.1.1 Integer binary search on the answer

Use when `ok(x)` is monotone: `false false ... false true true ...`. Write the invariant, not a formula you half-remember: **`lo` is always bad, `hi` is always good.**

```
// smallest x in (lo, hi] with ok(x) true; requires ok(hi) true, ok(lo) false (lo may be a sentinel)
template <class F> ll firstTrue(ll lo, ll hi, F ok) {
    while (hi - lo > 1) {
        ll mid = lo + (hi - lo) / 2;
        (ok(mid) ? hi : lo) = mid;
    }
    return hi;
}
// largest good x for "true ... true false ... false": lastTrue = firstTrue of !ok, minus 1
```

- Bounds: pick `lo` = something surely infeasible (often `-1` or `0`), `hi` = surely feasible (`1e18`, `2e9`). Check that `ok` never overflows at `hi` (e.g. `mid * cnt` with `mid = 1e18` → use `__int128` or early exit).
- **Maximize the minimum** (e.g. Hockey Fans, ACPC 2024: largest D such that M non-overlapping windows of length S all have values ≥ D): `ok(D)` = greedy scan left to right taking a window as soon as the last S values are all ≥ D. Monotone because a smaller D is easier.
- **Minimize the maximum** (split array into k parts minimizing max sum; minimize max edge on a path): `ok(X)` = greedy "how many parts do I need with cap X" ≤ k.
- **k-th smallest of an implicit set** (k-th smallest pairwise sum/difference, k-th fraction): binary search value v, count elements ≤ v (two pointers on sorted arrays is usually O(n)).
- **Binary search + graph** (Arachnophobia, Horse Snatchers): fix the parameter, then run BFS/Dijkstra on the allowed subgraph and check if the target is reachable within the time limit. Complexity × log(range).
- Searching over sorted *candidate* values (e.g. distinct edge weights) instead of the whole range keeps the answer exact and iterations ≤ log n.

2.1.2 Real-valued binary search

```
// sketch: ok(mid) = your feasibility test
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; ++it) {   // 100 halvings: 2^-100 relative; never loop on hi-lo>eps
    double mid = (lo + hi) / 2;
    if (ok(mid)) hi = mid; else lo = mid;
}
```

Fixed iteration count avoids infinite loops caused by precision. 60 iterations are enough for 1e-9 relative on [0, 1e9]; cost is usually negligible.

2.1.3 Ternary search (unimodal functions)

```
// maximum of strictly unimodal f on [lo, hi] (reals)
template <class F> double ternaryMax(double lo, double hi, F f) {
    for (int it = 0; it < 200; ++it) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1; else hi = m2;
    }
    return (lo + hi) / 2;
}
// integers: shrink while hi - lo > 2, then check lo..hi directly
template <class F> ll ternaryMaxInt(ll lo, ll hi, F f) {
    while (hi - lo > 2) {
        ll m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1 + 1; else hi = m2;
    }
    ll best = lo;
    for (ll x = lo; x <= hi; ++x) if (f(x) > f(best)) best = x;
    return best;
}
```

- Integer version fails with plateaus (f(m1) == f(m2) not at the max). If f can be flat, binary search on the sign of `f(x+1) - f(x)` instead (valid when f is concave: differences are non-increasing).

- Convex functions: sums of convex functions are convex (e.g. $\Sigma|x - a_i|$, Σ dist(P(t), Q_i) for linear motion). Distance between two points moving linearly over time is convex in t → ternary search on time.
- Nested ternary search works for functions convex in 2 variables (e.g. Fermat point, smallest enclosing circle centre) but is slow: 100×100 evaluations.

2.1.4 Parametric search / fractional programming

Maximize $\Sigma a_i / \Sigma b_i$ over feasible sets S ($b > 0$). The answer $\geq \lambda$ iff $\max_S \Sigma(a_i - \lambda b_i) \geq 0$. So binary search λ and solve the *unweighted* problem with weights $a_i - \lambda b_i$:

- Choose exactly k items: sort by $a_i - \lambda b_i$, take top k.
- Spanning tree minimizing Σcost/Σgain (ACPC 2023 *Buildin' Roads*): min ratio ⇔ binary search λ, MST with weights cost $- \lambda \cdot$gain, check total ≤ 0.
- Cycle with maximum mean (Σw/len): binary search λ, look for positive cycle with weights $w - \lambda$ (Bellman–Ford).

Dinkelbach converges faster: start $\lambda = 0$; repeat: $S = \text{argmax } \Sigma(a - \lambda b)$; $\lambda = \Sigma_S a / \Sigma_S b$; until no improvement (usually < 10 iterations).

2.1.5 Binary lifting search ("walk down powers of two")

When you can test "is the answer $\geq$ pos + 2^k" incrementally (Fenwick tree prefix walk, binary lifting on ancestors, sparse table), descend over k from high to low instead of a separate binary search — saves a log factor. See BIT `lower_bound` (§3.2) and LCA (§5.4).

2.1.6 Parallel binary search (offline, many queries)

Q queries each need "first time t when condition holds" where the structure evolves over time (e.g. first time u,v connected after edge additions; first time a sum reaches a threshold). Do log T rounds: in each round sort queries by their current `mid`, replay the events once from the start, and answer all `ok(mid)` checks along the way. Cost: O((T + Q) log T × cost of one event).

2.2 Prefix Sums, Difference Arrays, Sweep Line, Contribution

2.2.1 Prefix sums (1D/2D) and difference arrays

```
// 1D: p[i] = a[0] + ... + a[i-1]; sum of a[l..r] = p[r+1] - p[l]
// 2D: P[i][j] = sum of a[0..i-1][0..j-1]
struct Pref2D {
    vector<vector<ll>> P;
    Pref2D(const vector<vector<ll>>& a) {
        int n = a.size(), m = a[0].size();
        P.assign(n + 1, vector<ll>(m + 1, 0));
        rep(i, 0, n) rep(j, 0, m) P[i + 1][j + 1] = a[i][j] + P[i][j + 1] + P[i + 1][j] - P[i][j];
    }
    ll query(int r1, int c1, int r2, int c2) {  // inclusive rectangle
        return P[r2 + 1][c2 + 1] - P[r1][c2 + 1] - P[r2 + 1][c1] + P[r1][c1];
    }
};
```

- **Difference array:** add v to a[l..r] ⇒ `d[l] += v; d[r+1] -= v;` then prefix-sum d once. 2D: `d[r1][c1] += v; d[r1][c2+1] -= v; d[r2+1][c1] -= v; d[r2+1][c2+1] += v`.
- Adding an arithmetic progression to ranges: apply difference twice (second-order difference).
- **Count subarrays with sum = k:** iterate prefix p, `ans += cnt[p - k]; cnt[p]++` (hashmap; start with `cnt[0] = 1`). Same for xor (`p ^ k`), and for "sum divisible by m" (`cnt[p mod m]`).
- **Longest subarray with sum ≤ k (non-negative values)**: two pointers. With negatives: prefix + monotonic stack / sorted prefix minima + binary search.
- **Prefix of a function over a cycle** (rotations, circular arrays): double the array, take windows of length n.

2.2.2 Coordinate compression

```
// sketch: vals = every coordinate that matters
vector<ll> xs = vals;                       // all coordinates that matter
sort(all(xs)); xs.erase(unique(all(xs)), xs.end());
auto id = [&](ll x) { return int(lower_bound(all(xs), x) - xs.begin()); };
```

For intervals on a compressed axis, remember that compressed index i represents the *point* xs[i]; the gap between xs[i] and xs[i+1] has length `xs[i+1] - xs[i]` (needed for union-of-intervals length / area).

2.2.3 Sweep line

Turn geometry/intervals into **events sorted by coordinate**, process with a data structure.

- **Max overlapping intervals:** events (l, +1), (r, −1); sort; for closed intervals put +1 before −1 at equal coordinate.
- **Union length of intervals:** sort by l, merge greedily.
- **Area of union of rectangles:** sweep x; segment tree over compressed y storing "count covered > 0 length" (classic: node stores `cnt` and `covered len`, no push-down needed).
- **Pairs of intersecting segments / closest pair:** sweep with `set` ordered by current y (rarely needed at ACPC).
- **Offline queries:** sort queries together with updates by coordinate/time; answer each query when the sweep reaches it (e.g. "count points with x ≤ X and y ≤ Y" → sort by x, BIT on y).
- **Events at same coordinate:** decide the order explicitly (insertions before queries? removals first?) — this is the #1 sweep bug.

```
// sketch: max number of simultaneously active closed intervals [l, r]
vector<pair<ll,int>> ev;
for (auto [l, r] : iv) { ev.push_back({l, -1}); ev.push_back({r, +1}); } // -1 = open sorts first
sort(all(ev));
int cur = 0, best = 0;
for (auto [x, t] : ev) { cur -= t; best = max(best, cur); }
```

2.2.4 Contribution technique (sum over all subsets/subarrays/pairs)

Instead of enumerating structures, ask: **for each element, in how many structures does it contribute, and how much?**

- Σ over all subarrays of sum = Σ a_i · (i+1)·(n−i) (0-indexed).
- Σ over all subarrays of max: a_i is the max of subarrays with l ∈ (L_i, i], r ∈ [i, R_i) where L_i = previous index with value ≥ a_i (strict on one side, non-strict on the other to avoid double counting ties), R_i = next index with value > a_i. Contribution a_i·(i − L_i)·(R_i − i). Use monotonic stack (§2.3).
- Σ over pairs |a_i − a_j|: sort; a_i contributes `a_i·i − (sum of previous)`.
- Σ over all subsets of max: sort; a_i is the max in 2^i subsets.
- Σ of path lengths over all pairs in a tree: edge (parent, v) is used by sz[v]·(n − sz[v]) pairs.
- Expected value versions: same with probabilities (linearity of expectation, §4.5).

2.2.5 Offline tricks summary

- Answer queries in a convenient order (sorted by right endpoint, by threshold, by time) and restore original order at the end — store the query index.
- Reverse time for deletions (DSU can't delete, but can add).
- Divide & conquer over time / CDQ: split operations, handle "left half updates → right half queries" with a sort + BIT.

2.3 Two Pointers, Monotonic Stack, Monotonic Deque

2.3.1 Two pointers / sliding window

Valid when, for each left end `l`, the best/maximal right end `r(l)` is non-decreasing in `l` (window property is monotone: shrinking a valid window keeps it valid).

```
// longest window with at most k distinct values
int longestKDistinct(const vector<int>& a, int k) {
    unordered_map<int, int> cnt;
    int best = 0;
    for (int l = 0, r = 0; r < sz(a); ++r) {
        cnt[a[r]]++;
        while (sz(cnt) > k) { if (--cnt[a[l]] == 0) cnt.erase(a[l]); ++l; }
        best = max(best, r - l + 1);
    }
    return best;
}
```

- Count windows satisfying a monotone property: for each r, all l' ≥ l(r) work → `ans += r - l + 1`.
- "Exactly k" = atMost(k) − atMost(k−1).
- Two sorted arrays: pairs with a_i + b_j ≤ X — one pointer forward on a, one backward on b.
- **Does not work with negative numbers for sums** (window sum isn't monotone) — use prefix sums + sorted structure instead.

2.3.2 Monotonic stack: nearest greater / smaller

```cpp
// prv[i] = largest j < i with a[j] >= a[i] (or -1); nxt[i] = smallest j > i with a[j] > a[i] (or n)
// (>= on one side, > on the other: ties are counted exactly once in "subarrays where a[i] is max")
void nearestGreater(const vector<ll>& a, vector<int>& prv, vector<int>& nxt) {
    int n = sz(a); prv.assign(n, -1); nxt.assign(n, n);
    vector<int> st;
    rep(i, 0, n) {
        while (!st.empty() && a[st.back()] < a[i]) { nxt[st.back()] = i; st.pop_back(); }
        prv[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
}
```

Uses: largest rectangle in histogram (nearest smaller on both sides, area $h_i \cdot (R - L - 1)$); sum of subarray maxima/minima (contribution, §2.2); ACPC 2024 *Topographic Isolation* (for each peak, nearest index on the left and right with height ≥ peak: answer = min distance); visibility ("how many people can i see"); next greater element in a circular array (iterate 2n).

Maximal all-ones rectangle in a 0/1 grid: for each row keep `h[j]` = consecutive ones above; run histogram per row. O(nm).

2.3.3 Monotonic deque: sliding window min/max

```cpp
// mins[i] = min(a[i-k+1..i]) for i >= k-1
vector<ll> slidingMin(const vector<ll>& a, int k) {
    deque<int> dq; vector<ll> res;
    rep(i, 0, sz(a)) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

- **DP optimisation:** `dp[i] = min_{i-k ≤ j < i} dp[j] + c_i` → keep deque of j by dp value. Also `dp[i] = max_{j} (dp[j] - f(j)) + g(i)` with a sliding range of j.
- 2D sliding window min: apply 1D on rows, then on columns of the result.
- Shortest subarray with sum ≥ K with negative numbers: deque over prefix sums (increasing), pop front while `p[i] - p[front] ≥ K`.

2.3.4 Other amortised patterns

- **Stack simulation**: bracket matching, "remove adjacent equal pairs", evaluating expressions (shunting-yard).
- **Next pointer / "jump" with path compression**: "find next unvisited index ≥ i" — DSU where `nxt[i] = i+1` after visiting i; O(α) per query. Great for "paint ranges, each cell painted once" problems.
- **Small-to-large merging**: merge smaller set into larger → O(n log n) total insertions (§3.4).

2.4 Greedy and Exchange Arguments

A greedy is only as good as its proof. Standard proof: take an optimal solution that differs from greedy at the first position; **swap** to agree with greedy without making it worse. If you can't prove it, stress test against a brute force (§1.2) before submitting.

2.4.1 Finding the sort key by exchange

Two adjacent items i, j in an order; compare cost(…i j…) vs cost(…j i…). The difference only involves i and j ⇒ sort by that comparison.

problem	key
minimise Σ completion times (1 machine)	ascending processing time (SPT) — ACPC 2016 *Wood Cutting*
minimise $\Sigma w_i \cdot C_i$	ascending t_i / w_i, i.e. compare `t_i*w_j < t_j*w_i` (integers!)
minimise max lateness	ascending deadline (EDF)
concatenate strings to get smallest string	`a + b < b + a`
tasks with requirement r_i and cost c_i (need ≥ r to start, lose c)	descending $r_i - c_i$
two machines, each job on A then B (Johnson)	jobs with a<b first by a ascending, then others by b descending

Always compare with cross-multiplication in integers instead of dividing.

2.4.2 Interval greedies

- **Max number of disjoint intervals:** sort by right end, take if start > last end.
- **Min points to stab all intervals:** sort by right end; if interval not stabbed, put a point at its right end.
- **Min intervals to cover [0, L]:** sort by left; repeatedly among intervals with left ≤ covered, take the one with max right.
- **Partition intervals into min number of "rooms":** sort by start, min-heap of end times; answer = max overlap.
- **Assign intervals to k machines maximizing count:** sort by end, multiset of machine free times; assign to the machine with the latest free time ≤ start.

2.4.3 Deadline scheduling

Unit jobs with deadlines, maximise #jobs (or profit): sort by profit desc, place each job in the latest free slot ≤ its deadline (DSU "next free slot" trick, §3.1). Or: sort by deadline, push profits into a min-heap, pop the smallest whenever heap size > deadline.

Jobs with durations and deadlines, maximise count (Moore–Hodgson):

```
// jobs (deadline, duration); maximum number completed on time
int maxOnTime(vector<pair<ll,ll>> jobs) {
    sort(all(jobs));
    priority_queue<ll> pq; ll t = 0;
    for (auto [d, p] : jobs) {
        pq.push(p); t += p;
        if (t > d) { t -= pq.top(); pq.pop(); }   // drop the longest job so far
    }
    return sz(pq);
}
```

Feasibility check with deadlines (ACPC 2020 *Hired Help*: can we form k groups of K unit jobs?): a set of unit jobs is schedulable on c machines iff for every t, #jobs with deadline ≤ t is ≤ c·t. Combine with binary search on k and pick the jobs with the *latest* deadlines.

2.4.4 Heap greedies

- **Merge costs (Huffman):** always merge two smallest; total cost Σ.
- **Regret / "undo" greedy:** buy low sell high with one transaction per day: min-heap; for each price p: if `top < p` then `profit += p - top`, pop, and push p (this copy lets the sale be "undone" later); then always push p. Same idea: choose k non-adjacent elements with max sum — take max, replace it with (left + right − it) node, linked list.
- **k largest sums / k best combinations:** heap of states with "next" transitions (e.g. pairs (i, j) of two sorted arrays, push (i+1, j) and (i, j+1) with a visited set).

2.4.5 Other common greedy facts

- To maximise Σ a_i·b_{π(i)}: sort both same direction (rearrangement inequality); to minimise: opposite directions.
- Median minimises Σ|x − a_i|; mean minimises Σ(x − a_i)².
- Making an array non-decreasing with ±1 ops (min cost) → slope trick / median heap.
- Lexicographically smallest answer: build character by character, at each step try the smallest choice and check feasibility of completing (feasibility check is often a simple counting argument).
- Coin change greedy only works for canonical systems (1,5,10,25…); for arbitrary coins use DP.
- Pair the largest with the smallest (boats/teams with capacity): sort + two pointers.

2.5 Complete Search, Meet in the Middle, Square Root, Randomization

2.5.1 Brute force done right

- Enumerate subsets: `for (int m = 0; m < (1 << n); ++m)`. Permutations: sort, then `do { } while (next_permutation(all(p)));` (n ≤ 10: 3.6M perms).
- **Backtracking with pruning:** order choices (most constrained first), prune with an optimistic bound (current + best possible remaining ≤ best found ⇒ stop), memoize repeated states.
- **Iterate over the small dimension:** if one of the parameters is ≤ 20–200 (number of colours, max value, number of properties), loop over it explicitly.
- **Search the answer space instead of the input space** when the answer is small (e.g. answer ≤ 60 because values double).
- Before brute forcing, count states × transitions and compare with $3 \cdot 10^8$.

2.5.2 Meet in the middle

Split n ≤ 40 items into halves, enumerate $2^{n/2}$ each, combine with sort + two pointers/binary search/hash map.

```
// number of subsets with sum <= S (n <= 40)
ll countSubsets(const vector<ll>& a, ll S) {
    int n = sz(a), h = n / 2;
    auto gen = [&](int l, int r) {
        vector<ll> s = {0};
        for (int i = l; i < r; ++i) {
            int k = sz(s);
            rep(j, 0, k) s.push_back(s[j] + a[i]);
        }
        sort(all(s)); return s;
    };
    vector<ll> L = gen(0, h), R = gen(h, n);
    ll cnt = 0; int j = sz(R) - 1;
    for (ll x : L) {                        // L ascending -> pointer in R only moves down
        while (j >= 0 && x + R[j] > S) --j;
        cnt += j + 1;
    }
    return cnt;
}
```

Also: 4-sum = pair sums of two halves; bidirectional BFS on huge implicit graphs (puzzle states) — expand from both ends, stop when frontiers meet.

2.5.3 Square root decomposition

- **Blocks of size B ≈ √n:** each block stores an aggregate (+ lazy tag). Update/query touch O(n/B) full blocks + O(B) loose elements. Handy when the operation is weird (e.g. "add x to range, count elements < y in range": keep each block sorted → O(√n log n)).
- **Heavy/light by frequency:** values occurring > √n times are "heavy" (at most √n of them) → precompute for heavy, brute force for light.
- **Σ of sizes = n ⇒ at most O(√n) distinct sizes.** (Knapsack with Σ w = n has O(√n) distinct weights → bounded knapsack per distinct weight.)
- **Divisor/quotient blocks:** `n / i` takes O(√n) distinct values; iterate blocks:

```
// sum_{i=1}^{n} f(n / i) grouped: for each block [l, r], n / i == q
ll sumQuotients(ll n) {
    ll res = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll q = n / l; r = n / q;
        res += (r - l + 1) * q;     // replace q by f(q), (r-l+1) by the weight of [l, r]
    }
    return res;
}
```

- **Mo's algorithm** for offline range queries: §3.4.
- **Rebuild periodically:** buffer updates; every √q updates rebuild the static structure; queries check the static structure + the buffer.

2.5.4 Randomization

- **Random shuffle** before inserting into structures that degrade on sorted input (treap keys, BST, hull insertion).
- **Random sampling** to find a majority/frequent element: an element occupying ≥ 1/k of the array is hit with probability ≥ 1/k per sample; 30·k samples → failure ≈ e^{-30}.
- **Hashing with random base/mod** (§8.1); **xor-hashing of sets/multisets**: assign each value a random 64-bit number; set equality ⇔ xor (or sum) of hashes equal. Subarray is a permutation of 1..k ⇔ sum of hashes of subarray = prefix-hash of {1..k}.
- **Random restarts / simulated annealing** for optimization problems with no clear structure (last resort; tune by time limit with `clock()`).
- `mt19937_64 rng(seed)`; never `rand()` (max 32767 on some platforms).

2.5.5 Iterative deepening / search with memo

- Game/puzzle with small depth: IDDFS or BFS over states stored in hash map (encode state into a 64-bit integer).
- **Memoized recursion over states** = DP. If the state space is unclear, write the recursive brute force, then add a `map<State, Answer>` and see if it's fast enough.

2.6 Bit Manipulation and Bitset

2.6.1 Tricks

expression	meaning
`x & -x`	lowest set bit
`x & (x - 1)`	clear lowest set bit; `== 0` ⇔ x is a power of two (x > 0)
`__builtin_popcountll(x)`	number of set bits
`__builtin_ctzll(x)` / `__builtin_clzll(x)`	trailing / leading zeros (undefined for 0!)
`63 - __builtin_clzll(x)`	floor(log2 x) (`__lg(x)` also works)
`x >> k & 1`	k-th bit
`x ^ (1LL << k)`	toggle k-th bit
`(1LL << n) - 1`	mask of n ones
`a + b == (a ^ b) + 2 * (a & b)`	sum decomposes into xor + carries
`a ^ b ^ b == a`	xor cancels; prefix xor gives range xor

```cpp
// iterate all submasks of mask (descending), including 0: total over all masks is 3^n
void subs(int mask) {
    for (int s = mask; ; s = (s - 1) & mask) {
        // use s
        if (s == 0) break;
    }
}
// iterate all supersets of mask within n bits
void supers(int mask, int n) {
    for (int s = mask; s < (1 << n); s = (s + 1) | mask) { /* use s */ }
}
// next mask with the same popcount (Gosper's hack)
ll nextSamePop(ll x) { ll c = x & -x, r = x + c; return (((r ^ x) >> 2) / c) | r; }
```

2.6.2 XOR facts

- XOR of 0..n: `n % 4 == 0 ? n : n % 4 == 1 ? 1 : n % 4 == 2 ? n + 1 : 0`.
- Maximum xor of two numbers in a set: binary trie, greedily go to the opposite bit (O(30) per query).
- Maximum xor of any subset / k-th xor / "is x representable": **linear basis** (§7.4).
- Per-bit independence: many problems on sums of xors/ands/ors split into 30 independent problems, one per bit (count pairs with bit k set in a_i ^ a_j = cnt0·cnt1).
- `a | b` and `a & b`: `a + b = (a | b) + (a & b)`.

2.6.3 Bitset

`bitset<N>` operations are N/64 word ops — 64× speedup for boolean DP.

```cpp
// subset-sum reachability up to MAXS: O(n * MAXS / 64)
const int MAXS = 100001;
bitset<MAXS> reachable(const vector<int>& w) {
    bitset<MAXS> b; b[0] = 1;
    for (int x : w) b |= b << x;
    return b;
}
```

- Transitive closure / reachability in a DAG: `reach[v] |= reach[u]` in reverse topo order: O(n·m/64).
- Counting triangles: for each edge (u,v): `(adj[u] & adj[v]).count()`.
- Sets of reachable path lengths in a DAG (ACPC 2017 *Quantum Superposition*): `len[v] |= len[u] << 1`.
- O(n^2) DP where the transition is an OR/AND over a row (LCS-like boolean, interleaving check like ACPC 2018 *Homework*) can often be done row-by-row with shifts.
- `_Find_first()`, `_Find_next(i)` iterate set bits fast. Size must be a compile-time constant; for dynamic sizes pick the max or template over sizes.

2.6.4 Gray code

`g = i ^ (i >> 1)`: consecutive values differ in one bit — enumerate subsets toggling one element at a time (useful when add/remove of an item is O(1) but recomputation is expensive).